

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 1 — Fondamenti degli LLM

Genealogia degli LLM

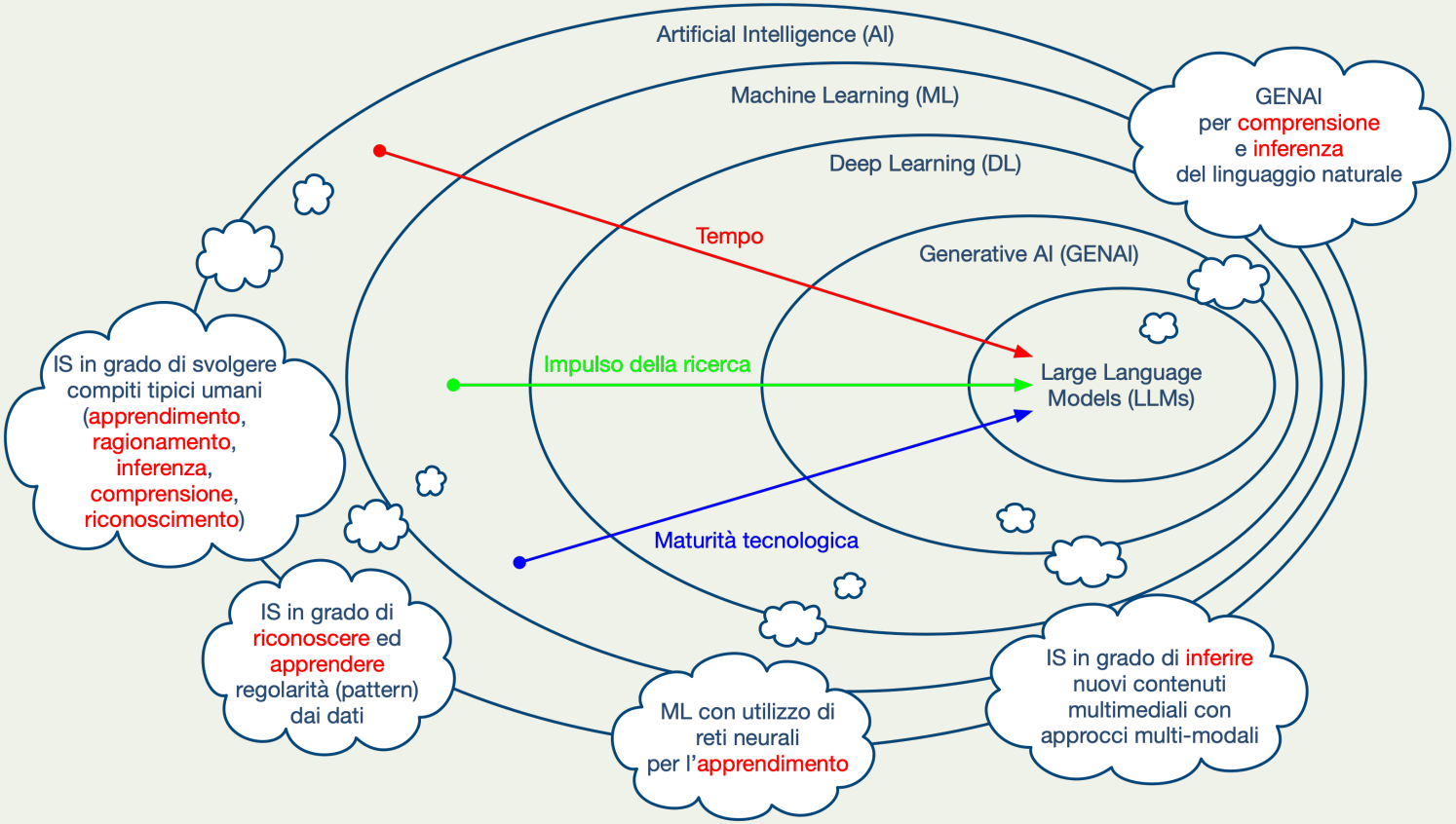
Dove siamo e che cosa serve

Modulo 1 di 9: il modello mentale che regge il resto del corso

- **L'obiettivo:** un **modello mentale condiviso**, così che tutti capiscano perché il resto del corso funziona come funziona
- **Cinque domande**, una per parte del modulo:
  1. Come si è arrivati agli LLM? **(questa parte)**
  2. Che cosa fanno, che cosa leggono, come imparano?
  3. Che cosa vedono: token e finestra di contesto
  4. Dove sono forti, dove sbagliano, e perché «allucinano»
  5. La famiglia Claude: i modelli del piano Pro — Opus, Sonnet, Haiku — e come scegliere
- **Poi la pratica:** lo stesso prompt su tre modelli, nel vostro Claude Code (esercitazione M1-E01)
- **Che cosa serve:** dai moduli precedenti, niente — si comincia da qui. Tenete aperto claude.ai, per due prove guidate; per l'esercitazione, Claude Code con `tools/verifica_ambiente.sh` tutto verde

Le scatole cinesi

Dall’AI agli LLM, un cerchio alla volta



Dall’intelligenza artificiale agli LLM: ogni cerchio contiene quelli più interni
© Simone Scannapieco

Language AI

… o NLP

- Lo scopo: far svolgere ai calcolatori **compiti utili con il linguaggio umano** — per esempio elaborare un testo, o generarne uno
- Un campo con **molti nomi**: *natural language processing* (**NLP**), linguistica computazionale e, di recente, anche soltanto *language modeling*
- Un manuale sugli LLM, *Hands-On Large Language Models* (O'Reilly), lo chiama **Language AI**
- **Trasversale** ai cerchi, dall'AI agli LLM: ci sono passate le regole scritte a mano, la statistica, le reti neurali — è il filo che seguiamo fino agli LLM

Language models

… o modelli del linguaggio

- Un modello del linguaggio è un modello di *machine learning* che **predice le parole che seguono**, dato il testo che le precede: il suo **contesto**
- Il più semplice si addestra **contando** le sequenze di parole in una raccolta di testi: gli *n-grammi*
- Poi i modelli **neurali**, molto più potenti, fino ai *large language model* di oggi, costruiti sul **Transformer** — lo vediamo alla fine di questa parte
- Come lo fa un LLM come Claude: nella parte «Cosa fanno, cosa leggono, come imparano»

Le origini: ELIZA (1966)

Il primo *chatbot* della storia

- Di **Joseph Weizenbaum**, al MIT: descritta su *Communications of the ACM* nel gennaio 1966, girava sul sistema *time-sharing* MAC
- Simulava uno **psicoterapeuta rogersiano**: cercava nella frase una **parola chiave** e trasformava la frase con la **regola** associata — regole **scritte a mano**
- **A voce, sì o no:** chi ci parlava si accorgeva di parlare con una macchina?
- **Non tutti.** Alcune persone erano «molto difficili da convincere» che ELIZA non fosse umana; e chi ci parlava si coinvolgeva: conversazioni molto personali, fino a chiedere a Weizenbaum di uscire dalla stanza mentre scriveva a ELIZA

> *ELIZA mostra, se non altro, quanto sia facile creare e mantenere l'illusione della comprensione […]*
>
> **Joseph Weizenbaum, 1966 (tradotto)**

Prova ELIZA

ELIZA: come funzionava

Trasformazioni grammaticali: i pronomi rigirati

Trasformazioni grammaticali: prima di rispondere, ELIZA rigira i pronomi della frase

Regole semplificate

```
me  ->  you
my  ->  your
```

Nel copione stampato nell'articolo la regola per «me» si legge `(ME = YOU)`

👤	*«Men are all alike.»*	*«Gli uomini sono tutti uguali.»*
>_	*«In what way»*	*«In che senso»*
👤	*«They're always bugging us about something or other.»*	*«Ci tormentano sempre per una cosa o per l'altra.»*
>_	*«Can you think of a specific example»*	*«Ti viene in mente un esempio preciso»*
👤	*«Well, **my** boyfriend made **me** come here.»*	*«Be', è il mio ragazzo che mi ha fatto venire qui.»*
>_	*«**Your** boyfriend made **you** come here»*	*«Il tuo ragazzo ti ha fatto venire qui»*

Joseph Weizenbaum, 1966

ELIZA: come funzionava

Template di risposta: la frase dell’utente dentro uno stampo

Template di risposta: uno stampo con un buco, e il pezzo della frase dell’utente va nel buco

	*«I am **BLAH**»*	*«Io sono BLAH»*
>_	*«How long have you been **BLAH**»*	*«Da quanto tempo sei BLAH»*

Joseph Weizenbaum, 1966

Vale **qualunque cosa voglia dire BLAH**: ELIZA non lo sa, e non le serve saperlo

ELIZA: come funzionava

Parole chiave: scomporre la frase, poi ricomporla

- **Regole di *pattern matching*:** una regola di **scomposizione** taglia la frase in pezzi attorno alle parole che conosce; una di **ricomposizione** costruisce la risposta con quei pezzi

Scomposizione e ricomposizione, come nell'articolo

```
frase:           It seems that you hate me
scomposizione:   (0 YOU 0 ME)
pezzi:           (1) It seems that  (2) you  (3) hate  (4) me
ricomposizione:  (WHAT MAKES YOU THINK I 3 YOU)
risposta:        What makes you think I hate you
```

- `0` = un numero qualsiasi di parole; 3 = il terzo pezzo, qui *hate*

👤	*«It seems that you **hate** me»*	*«Sembra che tu mi odi»*
>_	*«What makes you think I **hate** you»*	*«Che cosa ti fa pensare che io ti odi»*

Joseph Weizenbaum, 1966

ELIZA: come funzionava

Strategia di ripiego: quando non trova una parola chiave

Strategia di ripiego: se nella frase non c'è nessuna parola chiave, ELIZA risponde con una **frase senza contenuto**, oppure riprende una trasformazione di prima

«Please go on»	*«Vai avanti, per favore»*
«What does that suggest to you»	*«Che cosa ti fa venire in mente»*
«Do you feel strongly about discussing such things»	*«Ti preme parlare di queste cose»*

Joseph Weizenbaum, 1966

Parole chiave, regole, stampi, frasi di ripiego: **tutto scritto a mano**, nel copione

Bag-of-Words

Rappresentare il testo con i numeri

- Un’espressione con più di settant’anni: nel 1954 il linguista Harris scriveva che il linguaggio **«non è soltanto un sacchetto di parole»** (*a bag of words*)
- Il problema: un calcolatore lavora con i **numeri**, e un testo libero — **non strutturato** — va trasformato in numeri
- Il *bag-of-words* lo fa in modo semplice: tratta il testo come un **sacchetto** di parole, senza ordine, e le **conta**

Bag-of-Words: come funziona

Tokenizzazione, vocabolario, conteggio

1. **Tokenizzazione:** si taglia il testo in pezzi, i ***token***; qui i token sono **parole intere**, tagliate sugli spazi, e la punteggiatura si scarta
   - ⚠ Nota bene: un LLM taglia il testo in pezzi diversi, anche **più piccoli di una parola** — lo vediamo nella parte «Token e finestra di contesto»

$_1$ «Che bel cane che hai!»

$_2$ «Ma il mio gatto è più bello»

Token										
che	bel	cane	che	hai	ma	il	mio	gatto	è	più	bello

Bag-of-Words: come funziona

Tokenizzazione, vocabolario, conteggio

1. **Tokenizzazione:** si taglia il testo in pezzi, i ***token***; qui i token sono **parole intere**, tagliate sugli spazi, e la punteggiatura si scarta
   - ⚠ Nota bene: un LLM taglia il testo in pezzi diversi, anche **più piccoli di una parola** — lo vediamo nella parte «Token e finestra di contesto»
2. **Vocabolario:** le voci distinte, ridotte alla **radice** (*stemming*: bel, bello → bell-) o alla **forma base** (hai → avere, è → essere); ogni voce ha il suo **posto fisso**

$_1$ «Che bel cane che hai!» $_2$ «Ma il mio gatto è più bello»

Vocabolario: dieci voci

che	**bell-**	can-	avere	ma	il	mi-	gatt-	essere	più

Bag-of-Words: come funziona

Tokenizzazione, vocabolario, conteggio

1. **Tokenizzazione:** si taglia il testo in pezzi, i ***token***; qui i token sono **parole intere**, tagliate sugli spazi, e la punteggiatura si scarta
   - ⚠ Nota bene: un LLM taglia il testo in pezzi diversi, anche **più piccoli di una parola** — lo vediamo nella parte «Token e finestra di contesto»
2. **Vocabolario:** le voci distinte, ridotte alla **radice** (*stemming*: bel, bello → bell-) o alla **forma base** (hai → avere, è → essere); ogni voce ha il suo **posto fisso**
3. **Conteggio:** per ogni testo, quante volte compare ogni voce del vocabolario

d_1 «Che bel cane che hai!» d_2 «Ma il mio gatto è più bello»

Vocabolario

	che	**bell-**	can-	avere	ma	il	mi-	gatt-	essere	più
d_1	2	**1**	1	1	0	0	0	0	0	0
d_2	0	**1**	0	0	1	1	1	1	1	1

Bag-of-Words

Limitazioni

Perdita dell'ordine: frasi di significato opposto possono avere **lo stesso sacchetto**

d_1 *«Il cane morde l'uomo»*
d_2 *«L'uomo morde il cane»*

Problema dell'ordine delle parole

	Vocabolario				
	il	can-	mordere	l-	uom-
d_1	1	1	1	1	1
d_2	1	1	1	1	1

Due righe identiche: **chi morde chi**, il sacchetto non lo sa

Bag-of-Words

Limitazioni

- **Perdita dell'ordine:** frasi di significato opposto possono avere **lo stesso sacchetto**
- **Mancanza di semantica:** non cattura il significato delle parole; **nessuna generalizzazione**: i sinonimi sono voci **del tutto separate**

- *«Re» e «Regina» dovrebbero essere correlati…*
- *Sinonimi come «felice» e «gioioso» dovrebbero avere una rappresentazione simile…*
- *E «pitone» e «serpente», l'uno un caso particolare dell'altro?*

Mancanza di relazioni semantiche

⚠ Per il sacchetto sono **sei voci diverse**, e basta

Bag-of-Words

Limitazioni

- **Perdita dell'ordine:** frasi di significato opposto possono avere **lo stesso sacchetto**
- **Mancanza di semantica:** non cattura il significato delle parole; **nessuna generalizzazione**: i sinonimi sono voci **del tutto separate**
- **Alta dimensionalità:** il vettore di un testo è lungo quanto il vocabolario, e **quasi tutto di zeri**

$_1$ *«Che bel cane che hai!»* $= (2, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, \ldots, 0)$

Problema della dimensionalità

⚠ Con un vocabolario grande come quello dei vettori pubblicati da Google, **3 milioni** di parole ed espressioni, questa frase diventerebbe una riga di **3 milioni di numeri**: tutti zero, tranne quattro

Word Embeddings

Word2Vec (2013) e le rappresentazioni dense

- **Word2Vec** (Mikolov et al., Google, 2013): ogni parola diventa un **vettore di numeri**, imparato dai testi
- Gli ***embedding***: vettori **corti e densi**, di lunghezza fissa — **300 numeri** per parola nei vettori pubblicati da Google, non milioni di colonne quasi tutte a zero
- Imparati da **raccolte di testi enormi**: Google News, circa **100 miliardi di parole**, per i vettori di Google; Wikipedia in inglese, English Gigaword, British National Corpus per altri modelli pubblici
- Word2Vec usa **apposta** modelli più semplici di una rete neurale: meno precisi ma, scrivono gli autori, addestrabili su molti più dati
- Quei numeri sono i **pesi** del modello: l'addestramento li regola finché le previsioni si avvicinano alle risposte giuste. Nelle reti neurali, i numeri imparati così si chiamano **parametri**
- Gli *embedding* li riprendiamo nel **Modulo 7**

Word2Vec: come funziona

Apprendimento delle relazioni tra parole

- **Principio:** parole che compaiono in **contesti simili** tendono ad avere **significati simili** — l'*ipotesi distributiva*, formulata negli anni Cinquanta da linguisti come Joos, Harris e Firth

$_1$ «[…] Il mio cane ama dormire nella sua cuccia […]»
$_2$ «[…] Il veterinario ha deciso di sterilizzare il gatto […]»
$_3$ «[…] Giocando, il mio cane si è fatto male e l'ho dovuto portare dal veterinario […]»
$_4$ «[…] Ho comprato una cuccia per il mio gatto, ma continua a preferire il divano! […]»

> *You shall know a word by the company it keeps.*
>
> **J. R. Firth, 1957**

Word2Vec: come funziona

Apprendimento delle relazioni tra parole

- **Principio:** parole che compaiono in **contesti simili** tendono ad avere **significati simili** — l'*ipotesi distributiva*, formulata negli anni Cinquanta da linguisti come Joos, Harris e Firth
- **Addestramento:** una domanda sì o no — la parola *c* compare **vicino** alla parola *w*? I pesi che servono a rispondere diventano gli *embedding*

$_1$ «[…] Il mio **cane** ama dormire nella sua **cuccia** […]»
$_2$ «[…] Il **veterinario** ha deciso di sterilizzare il **gatto** […]»
$_3$ «[…] Giocando, il mio **cane** si è fatto male e l'ho dovuto portare dal **veterinario** […]»
$_4$ «[…] Ho comprato una **cuccia** per il mio **gatto**, ma continua a preferire il divano! […]»

- Nelle frasi, «cane» e «gatto» hanno gli **stessi vicini**, «cuccia» e «veterinario»: a forza di sì e di no, l'addestramento rende **simili i loro vettori**

Word2Vec: come funziona

Apprendimento delle relazioni tra parole

- **Principio:** parole che compaiono in **contesti simili** tendono ad avere **significati simili** — l'*ipotesi distributiva*, formulata negli anni Cinquanta da linguisti come Joos, Harris e Firth
- **Addestramento:** una domanda sì o no — la parola *c* compare **vicino** alla parola *w*? I pesi che servono a rispondere diventano gli *embedding*
- **Risultato:** parole di significato simile, **vettori vicini**; e perfino le analogie

$$\text{vettore}(\textit{King}) - \text{vettore}(\textit{Man}) + \text{vettore}(\textit{Woman}) \longrightarrow \text{il più vicino è vettore}(\textit{Queen})$$

- Nessuno ha scritto a Word2Vec che «cane» e «gatto» si somigliano, né che il re sta all'uomo come la regina alla donna: l'ha **ricavato dai testi**. Le regole di ELIZA, invece, le aveva **scritte a mano** Weizenbaum

L'AI moderna non è programmata: è addestrata.

Prova WebVectors

Word2Vec

Limitazioni

- **Polisemia:** un **solo vettore** per parola, fisso, qualunque sia il contesto — anche per le parole che hanno più significati

> *«Che **cannonata**!» — un colpo di cannone, o un tiro fortissimo in una partita di calcio?*
>
> **Problema della polisemia**

- Per Word2Vec, **un vettore solo** per tutti e due i sensi

Word2Vec

Limitazioni

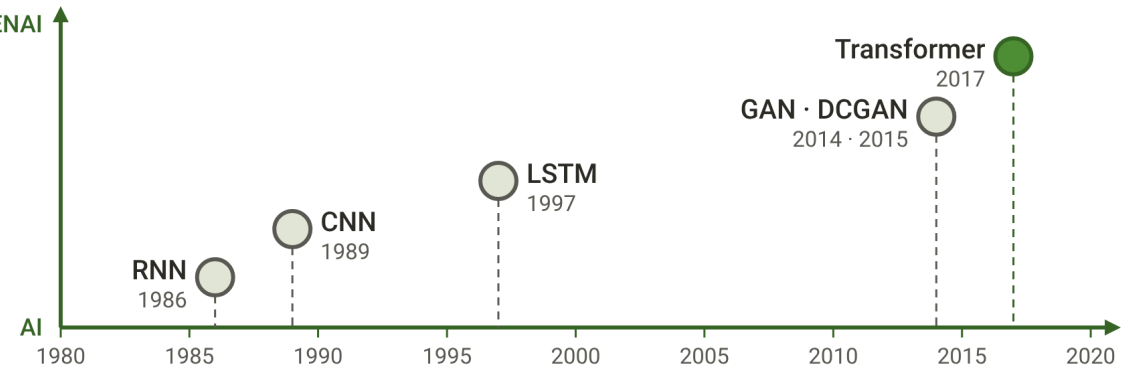
- **Polisemia:** un **solo vettore** per parola, fisso, qualunque sia il contesto — anche per le parole che hanno più significati
- **Contesti insufficienti:** una parola mai vista non ha un vettore, e una parola rara compare in poche frasi: da pochi contesti c'è **poco da imparare**

> *Che rappresentazione hanno parole come «supercazzola», «scappellamento», «antani»?!*
>
> **Problema dei contesti insufficienti**

- In un corpus dove compaiono in poche frasi, parole come queste — dal tormentone di *Amici miei* (1975), il film di Mario Monicelli — lasciano a Word2Vec **pochi contesti** da cui imparare

Le architetture prima del Transformer

schema illustrativo



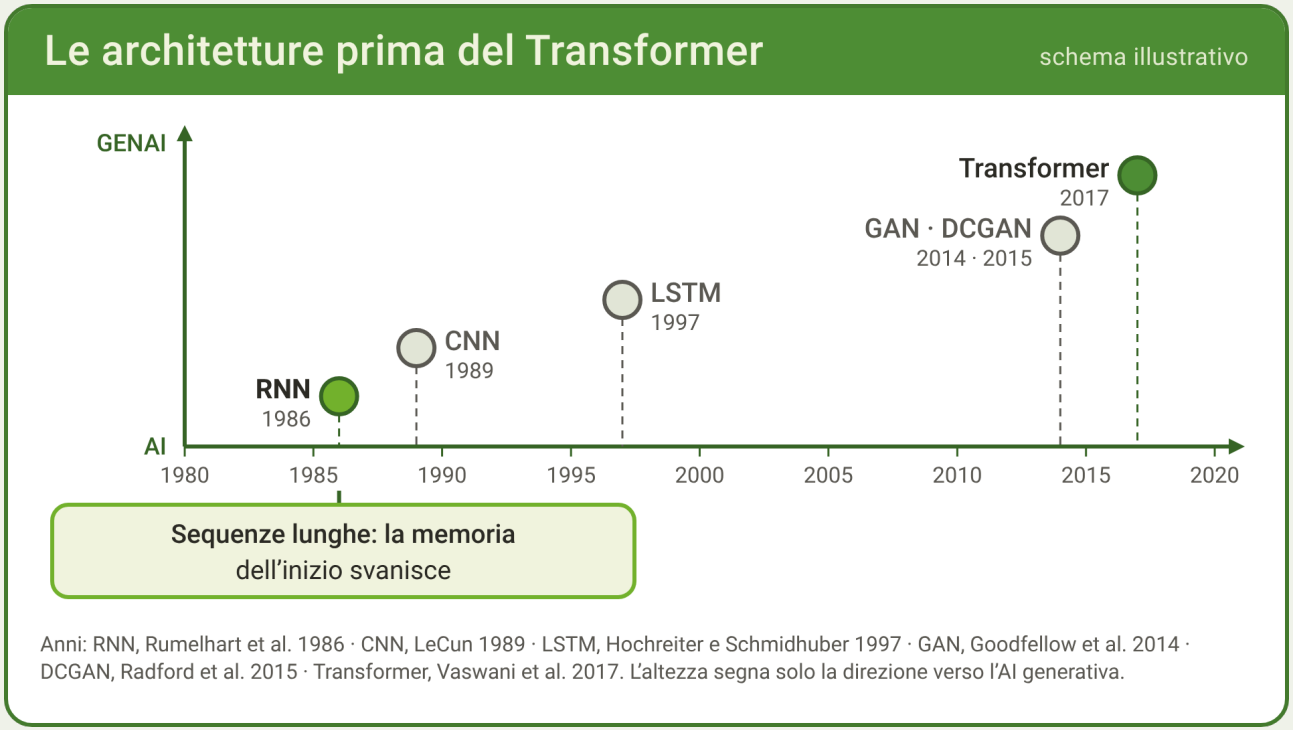
Anni: RNN, Rumelhart et al. 1986 · CNN, LeCun 1989 · LSTM, Hochreiter e Schmidhuber 1997 · GAN, Goodfellow et al. 2014 · DCGAN, Radford et al. 2015 · Transformer, Vaswani et al. 2017. L'altezza segna solo la direzione verso l'AI generativa.

Le architetture prima del Transformer: gli anni dalle fonti in calce alla figura
Schema illustrativo
© Simone Scannapieco — ridisegnato per questo corso

- Prima del Transformer, **famiglie diverse di reti neurali**: ognuna con un suo **limite**

GENAI

Limitazioni e contro in NLP



Le architetture prima del Transformer: gli anni dalle fonti in calce alla figura
Schema illustrativo
© Simone Scannapieco — ridisegnato per questo corso

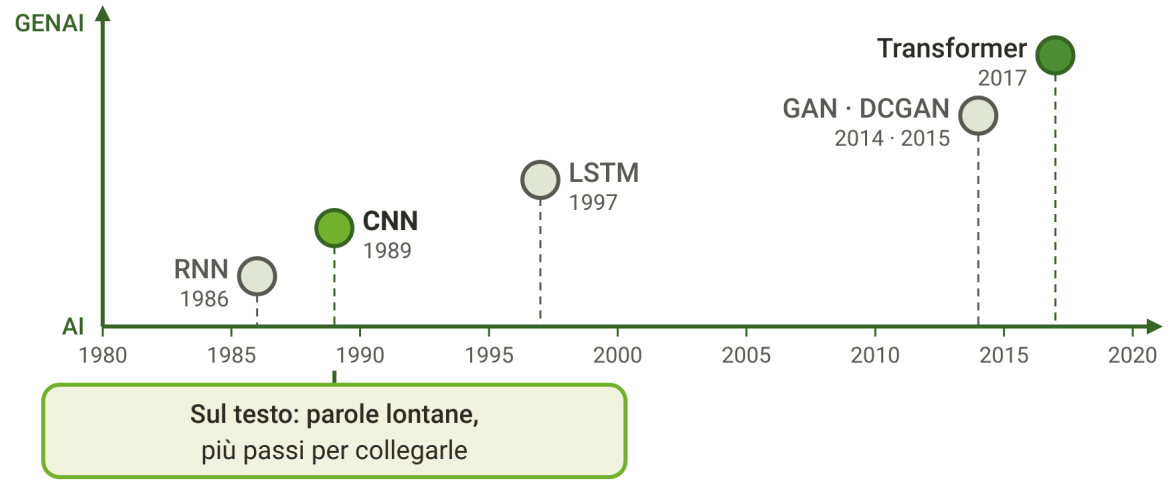
- **RNN**, reti ricorrenti (Rumelhart et al., 1986): leggono una sequenza un pezzo dopo l'altro, con una memoria di ciò che hanno letto; sugli **ingressi lunghi** la memoria dell'inizio svanisce

GENAI

Limitazioni e contro in NLP

Le architetture prima del Transformer

schema illustrativo



Anni: RNN, Rumelhart et al. 1986 · CNN, LeCun 1989 · LSTM, Hochreiter e Schmidhuber 1997 · GAN, Goodfellow et al. 2014 · DCGAN, Radford et al. 2015 · Transformer, Vaswani et al. 2017. L'altezza segna solo la direzione verso l'AI generativa.

Le architetture prima del Transformer: gli anni dalle fonti in calce alla figura
Schema illustrativo
© Simone Scannapieco — ridisegnato per questo corso

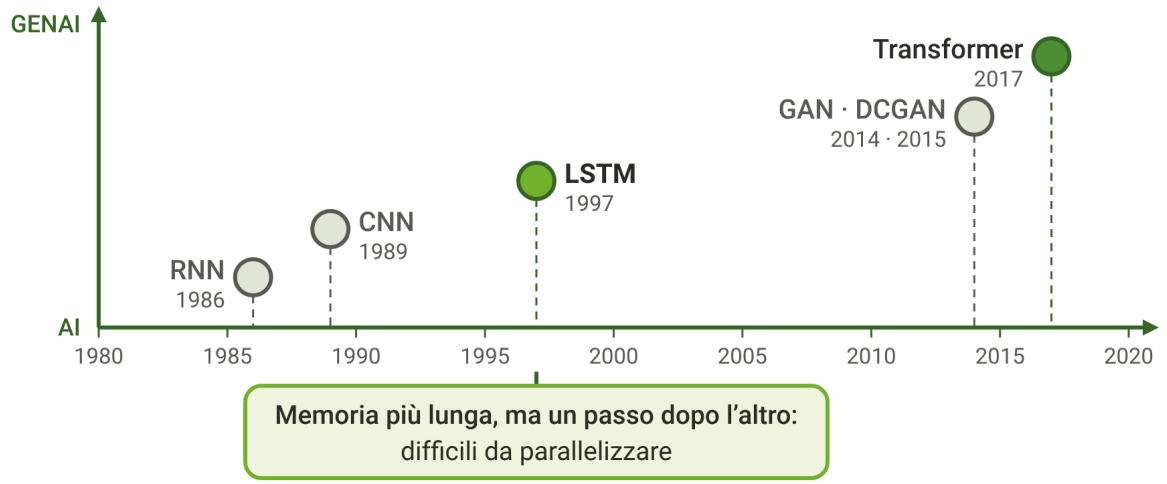
- **CNN**, reti convoluzionali (LeCun, 1989): nate per i **dati a griglia**, come le immagini; sul testo, collegare parole lontane richiede più passi, e le **dipendenze a distanza** sono più difficili da imparare

GENAI

Limitazioni e contro in NLP

Le architetture prima del Transformer

schema illustrativo



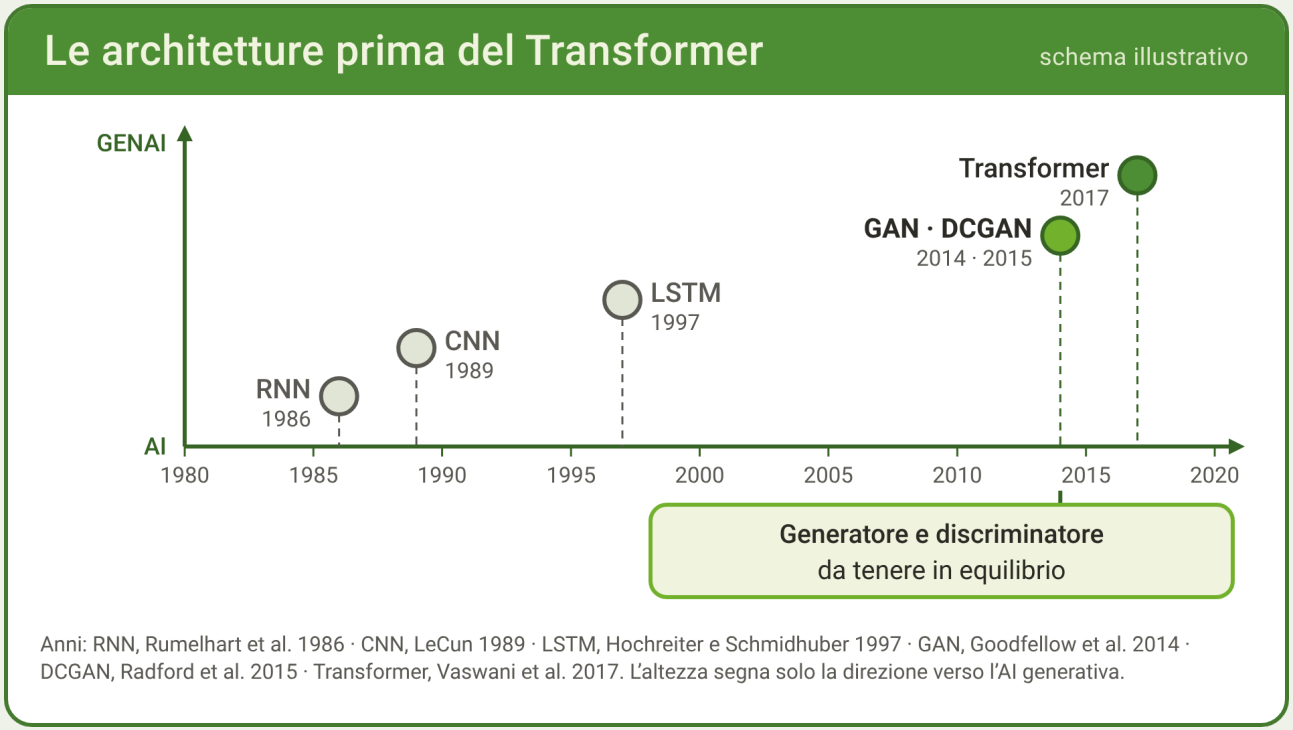
Anni: RNN, Rumelhart et al. 1986 · CNN, LeCun 1989 · LSTM, Hochreiter e Schmidhuber 1997 · GAN, Goodfellow et al. 2014 · DCGAN, Radford et al. 2015 · Transformer, Vaswani et al. 2017. L'altezza segna solo la direzione verso l'AI generativa.

Le architetture prima del Transformer: gli anni dalle fonti in calce alla figura
Schema illustrativo
© Simone Scannapieco — ridisegnato per questo corso

- **LSTM**: una rete ricorrente con **porte** che decidono che cosa ricordare e che cosa dimenticare — memoria più lunga, ma il calcolo resta un passo dopo l'altro: addestramento **difficile da parallelizzare**

GENAI

Limitazioni e contro in NLP

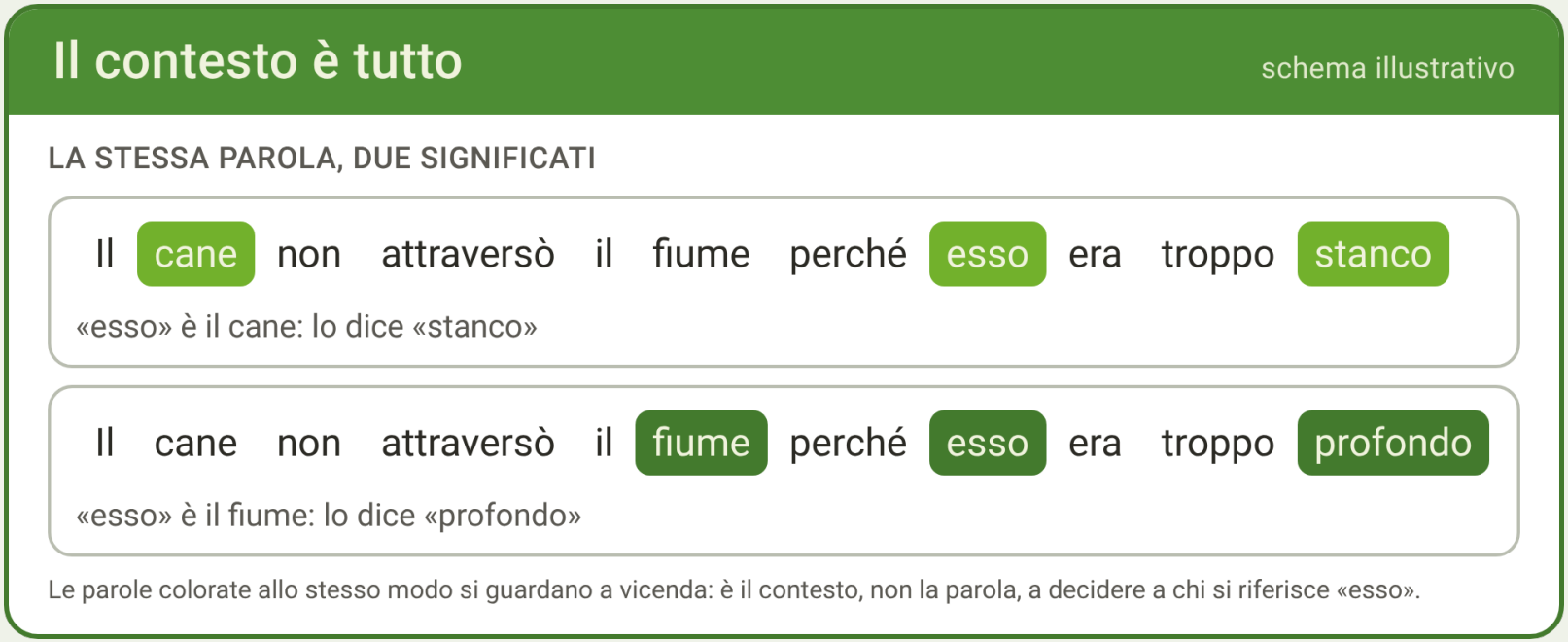


Le architetture prima del Transformer: gli anni dalle fonti in calce alla figura
Schema illustrativo
© Simone Scannapieco — ridisegnato per questo corso

- **GAN**: due reti in gara, un **generatore** e un **discriminatore**, da tenere in equilibrio; le GAN sono **instabili** da addestrare, e la DCGAN (2015) ne è la versione convoluzionale

Il contesto è tutto

La stessa parola, due significati — e li decide il resto della frase



«Esso» in due frasi quasi uguali: le parole dello stesso colore si guardano a vicenda
Schema illustrativo
© Simone Scannapieco

- Un vettore per parola non basta: il significato di alcune parole dipende da quello di altre — le **parole contestuali**
- Voi lo risolvete all'istante, senza accorgervene… ma una macchina, che vede solo **token in fila**?

La vecchia porta la sbarra

Quale lettura avete fatto per prima? Ditelo a voce

«La vecchia porta la sbarra»

Lettura 1

- **la vecchia** = una signora anziana
- **porta** = verbo: trasporta
- **la sbarra** = un'asta di ferro

Lettura 2

- **la vecchia porta** = una porta vecchia
- **la** = pronome: lei
- **sbarra** = verbo: blocca

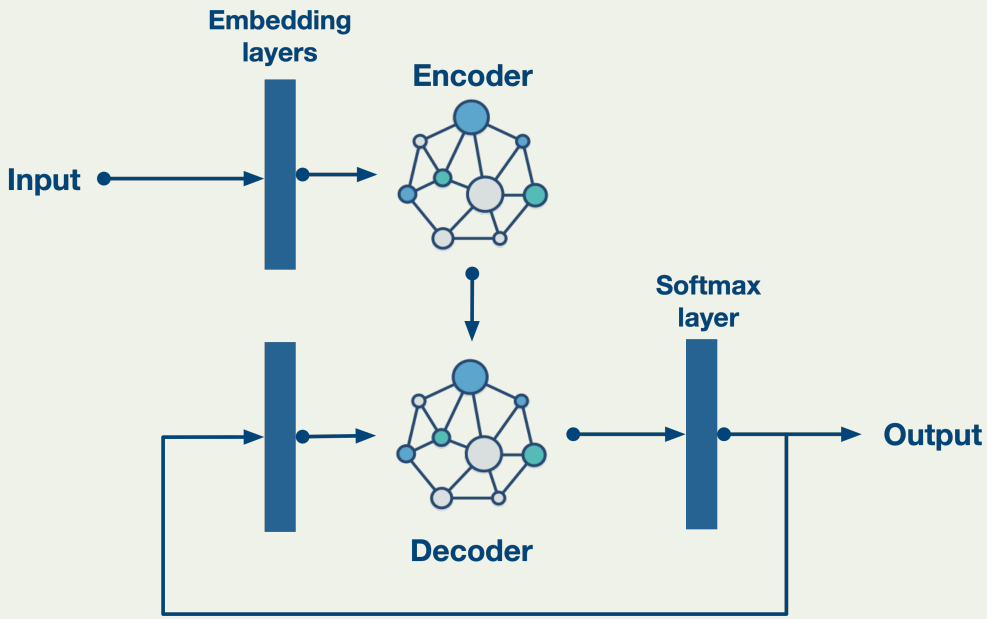
⚖ Stesse parole, stesso ordine: decide **quanto conta ogni parola per le altre**

🖧 È l'idea dell'**attention**: ogni token «guarda» gli altri token del contesto e pesa quanto contano, per decidere che cosa significa *lì*

Ogni pezzo del testo guarda gli altri: il significato lo decide il contesto.

Transformer (2017)

Dall’articolo «Attention Is All You Need»: chi legge, chi scrive, chi sceglie



Architettura Transformer
© Simone Scannapieco

- ***Encoder*** — legge l’*input* e ne costruisce una rappresentazione, pesando ogni pezzo sugli altri: l’attention
- ***Decoder*** — usa quella rappresentazione come **contesto** e genera l’*output* un pezzo alla volta: ogni pezzo scritto rientra come contesto del successivo
- ***Softmax*** — trasforma i punteggi in probabilità per il pezzo successivo
- ***Embedding*** — all’ingresso ogni pezzo diventa un vettore appreso, come in Word2Vec

Attention

Perché il Transformer ha vinto

- **Solo attention**: il Transformer rinuncia del tutto alla ricorrenza delle RNN e delle LSTM (un pezzo dopo l'altro) e alle convoluzioni delle CNN (nate per i dati a griglia, come le immagini)
- Ogni pezzo del testo pesca il contesto **direttamente dagli altri** — nell'encoder da tutti, nel decoder da quelli che lo precedono — invece di riceverlo di pezzo in pezzo
- Si calcola **in parallelo**: per questo è **molto più rapido da addestrare**
- Ed è ancora in servizio: il più grande Llama 3 di Meta (2024) è un Transformer da **405 miliardi** di parametri

Fin qui, da dove vengono i modelli come Claude.

Nella parte successiva: che cosa fanno, che cosa leggono, come imparano.